



PRIZMATIC

Agent-Authoritative Project Management Service

Live Demo: <https://dev.prismatic.app>

Admin Website: <https://admin.prismatic.app>

Team Members:

Jeongmin Sohn(jeongmin.sohn@stonybrook.edu),

Heejoon Yi(heejoon.yi@stonybrook.edu)

Table of Contents

1. Introduction.....	3
1.1 Product Description.....	3
1.2 Scope.....	3
1.3 Users.....	4
1.4 User Feedback.....	5
1.5 Existing Alternatives.....	6
1.6 Definitions.....	7
2. Requirements.....	7
2.1 Functional Requirements.....	7
2.2 Use Cases.....	11
2.3 Non-functional Requirements.....	15
3. System Architecture.....	16
3.1 Overview.....	16
3.2 Data Design.....	25
3.3 UML Sequence Diagrams.....	31
3.4 API Design.....	33
3.5 Deployment.....	33
3.6 Code Conventions.....	33
4. Schedule.....	36
5. Demo Screenshots.....	39

1. Introduction

1.1 Product Description

Prismatic is a web-based project management platform designed for small software development teams, including a single developer. It provides an AI project manager to support efficient project management and aims to minimize inefficiencies in work distribution, progress tracking, and code collaboration processes that occur within a team.

Many small teams struggle with organizing work, tracking progress, and managing pull requests across multiple tools due to the absence of a dedicated project manager. As a result, teams must manually monitor their project status and progress, often placing the burden on their most capable developers to act as de facto project managers. This significantly reduces their effectiveness in core development tasks, and the impact is even greater in smaller teams where the contribution of a single skilled developer is critical, ultimately leading to inefficient workflows and decreased productivity.

Prismatic addresses these problems by providing a centralized platform where teams can manage project progress, work items, assignments, priorities, sprints, and due dates. The AI project manager analyzes project context, team roles, workload, sprint state, and Work Item activity to decompose tasks, recommend priorities, and assign appropriate team members. Through GitHub integration, Prismatic can also identify relevant project context using work item codes in pull request titles and automatically provide AI-assisted pull request reviews.

By delegating selected project management tasks to the AI project manager, Prismatic reduces the need for manual progress tracking and enables more systematic project management, allowing developers to focus more on development work.

1.2 Scope

In Scope

This project focuses on developing a web-based AI-powered project management platform designed for small software development teams and individual developers.

The system includes:

- Centralized project and work item management

- Collaboration features such as workspace, member, comment, notification, and assignment management
- Hierarchical work item organization and sprint-based workflows
- AI-assisted features such as work item decomposition, backlog refinement, priority recommendations, task assignment, and pull request reviews
- GitHub integration for repository connections, work item identification, and automated pull request reviews

The system is deployed as a web application and is accessible via modern desktop browsers, such as Chrome, Safari, and Microsoft Edge.

Out of Scope

The following features are excluded from the current project scope due to time constraints:

- Native mobile application support
- Offline support
- AI coding workers and IDE integrations

1.3 Users

The intended users of Prismatic are individual developers and small software development teams. Users may have varying levels of technical and project management experience but are generally familiar with basic software development workflows and tools such as GitHub.

1. Individual Developers

- Students, freelancers, and independent developers working on personal, academic, or commercial projects
- Familiar with basic development tools and workflows
- May have limited project management experience and benefit from AI-assisted guidance
- Prefer intuitive interfaces and workflows that require minimal setup

2. Small Development Teams

- Teams typically consisting of 2–6 developers
- Members may have different responsibilities and levels of experience
- Familiar with development workflows such as pull requests, work item tracking, and sprint planning

- Require collaboration and coordination tools, but often lack a dedicated project manager

Usability Considerations

- The system should remain accessible to users with limited project management experience
- Interfaces and workflows should be clear and consistent to reduce unnecessary complexity
- AI-generated recommendations and actions should be presented transparently so users can understand and review them

1.4 User Feedback

User feedback was collected and considered throughout the development process to ensure that Prismatic addressed the practical needs of individual developers and small development teams.

1. Design Stage

During the design stage, the team presented initial UI mockups and core feature concepts to fellow software development students. Their feedback was used to evaluate the clarity of the product concept, navigation structure, and primary project management workflows.

2. Development Stage

During development, potential users tested early versions of the application by completing tasks such as creating workspaces, managing members, organizing work items, and using sprint workflows. Their feedback identified usability issues involving navigation, form interactions, and the presentation of Work Item information. In response, the team refined workspace and project navigation, simplified creation workflows, and improved the layout and metadata controls of Work Item pages.

3. Beta Release Stage

During the beta release stage, the team focused on identifying and resolving bugs through internal integration testing and limited user testing. This process revealed issues involving authentication error handling, notification delivery, Agent execution failures, deleted workspace and project navigation, and frontend-backend state synchronization. The team reproduced and analyzed the reported issues, implemented fixes across the frontend, backend, and Agent

runtime, and performed regression testing to verify that the fixes did not affect existing workflows. These efforts improved the reliability and stability of the final release.

4. Post-Release Stage

The team has recruited software developers and small development teams who have agreed to evaluate Prismatic using real project workflows after the final release. Their feedback will be collected to identify practical usability issues, improve the overall UI/UX, and guide the continued development of the platform.

1.5 Existing Alternatives

Several existing tools support project management for software development teams, including Jira and Notion. These platforms provide useful features, but they often require users to manually organize tasks, configure workflows, and track project progress. In contrast, Prismatic provides AI-assisted project management support to reduce manual coordination work.

1. Jira

- **Strengths:**
 - Highly customizable workflows and issue tracking
 - Strong support for agile methodologies, including sprints and backlog management
 - Widely used in professional software development teams
- **Weaknesses:**
 - Can be complex and overwhelming for small teams or beginners
 - Requires significant setup and workflow configuration
 - Many features may be unnecessary for simple or student-led projects
 - Project management decisions still depend heavily on manual planning and configuration

2. Notion

- **Strengths:**
 - Flexible and easy-to-use interface
 - Supports note-taking, documentation, and basic task management
 - Highly customizable for different use cases
- **Weaknesses:**

- Lacks structured software project management features compared to dedicated development tools
- Requires manual organization and maintenance
- Provides limited automation for development workflows
- Is not designed specifically around software engineering workflows such as pull requests, sprint planning, and work item tracking

Overall, Jira is powerful but can be too complex for small teams, while Notion is flexible but often requires more manual organization for structured software development. Prismatic aims to fill this gap by combining accessible project management workflows with AI-assisted support for work item organization, sprint planning, task assignment, and pull request reviews.

1.6 Definitions

This section defines key terms and concepts used throughout this document.

Workspace: A shared environment where a group of users collaborates on projects.

Project: A collection of related tasks within a workspace.

Work Item: A unit of work in the system, such as an epic, story, or task, used to organize and track progress.

Task: An actionable unit of work that can be completed within a short period of time.

Sprint: A fixed time period during which a set of work items is completed

Kanban Board: A visual board that displays work items in different stages (To Do, In Progress, In Review, Done).

Backlog: A prioritized list of work items that need to be completed in a project

Pull Request (PR): A request to merge code changes into a repository.

OAuth: An authentication method that allows users to log in using third-party accounts, such as Google and GitHub.

2. Requirements

2.1 Functional Requirements

Authentication & Account System

Users can create and manage accounts using email/password or OAuth (GitHub, Google).

- Users can sign up and log in

- The system ensures that each email address is unique
- The system allows users to securely authenticate
- Users can log out of the system
- Users can reset and change their passwords

User Profile

Users can view and update their profile information.

- Users can view their profile information, including their name, email, username, and authentication method
- Users can update their name
- Email/password users can change their password.

Workspace Management

Users can participate in and access workspaces as the primary organizational unit.

- Users can create a workspace
- The system supports role-based access (Owner, Admin, Member, Viewer)
- Workspace owners and admins can invite members via email or in-app notifications.
- Workspace Owners and Admins can update workspace settings and manage members.
- Workspace Owners and Admins can create, update, delete, and assign customized jobs.
- Workspace Owners can transfer ownership and delete the workspace.
- Users can leave workspaces they do not own.
- Users can have multiple jobs.

Project Management

Projects are created and managed within a workspace, serving as containers for organizing work, tasks, and team collaboration.

- Authorized users can create, view, update, and delete projects.
- Each project provides a dashboard for organizing and tracking Work Items.
- Projects can be connected to GitHub repositories.
- Project deletion safely redirects affected users away from unavailable project pages.

Work Item System

Work Items are created and managed within a project, serving as units for organizing, tracking, and executing tasks.

- Users can create, view, update, delete, restore, and permanently delete Work Items.
- Users can organize Work Items hierarchically using parent and child Work Items.
- Each Work Item has a human-readable identifier, such as PRSM-001.
- Users can update Work Item statuses, including Todo, In Progress, In Review, and Done.
- Users can set Work Item priorities, start dates, and due dates.
- Users can assign one or more workspace members to a Work Item.
- Users can reorder Work Items within the project dashboard.
- Users can comment on Work Items and mention workspace members.
- Users can associate documents with Work Items.

Sprint System

The system supports sprint-based workflow management.

- Users can create, view, update, and delete sprints.
- Users can define sprint names, goals, start dates, due dates, and statuses.
- Users can add and remove Work Items from sprints.
- Users can view the Work Items included in each sprint and their current statuses.
- Users can organize and review sprints according to their lifecycle status.

GitHub Integration

The system integrates with GitHub repositories.

- Authorized users can connect and disconnect GitHub repositories.
- The system receives supported pull request events through GitHub webhooks.
- The system can identify a related work item using a work item code included in a pull request title.
- The system associates pull request review activity with the relevant workspace, project, and work item when sufficient context is available.
- The system automatically triggers AI-assisted reviews for supported pull request events.
- The system can post review summaries and line-anchored comments to GitHub pull requests.

AI Pull Request Review

The system provides AI-assisted pull request reviews.

- The system analyzes the files and code changes included in a pull request.
- The system generates a concise summary of its overall assessment.
- The system identifies potential correctness, security, and contract-breaking issues.
- The system provides grounded improvement suggestions through line-anchored review comments.
- The system submits reviews using GitHub review outcomes, including COMMENT, APPROVE, and REQUEST_CHANGES.
- The system validates the pull request version and comment locations before submitting a review.

AI Project Manager

The system provides intelligent project management assistance.

- Users can request the AI project manager to generate Work Items for a feature.
- The AI project manager can decompose features and stories into structured parent and child Work Items.
- The AI project manager can detect duplicate or similar Work Items.
- The AI project manager can refine the project backlog.
- The AI project manager can recommend or apply appropriate Work Item assignments.
- The AI project manager can assist with sprint planning and project-risk analysis.
- Users can inspect AI runs, execution steps, actions, statuses, and completion times.
- The system records AI actions and prevents duplicate action execution.

Dashboard & Notifications

- Project dashboards organize Work Items by status and support progress tracking.
- Users can quickly update Work Item details from the project dashboard.
- The system provides real-time in-app notifications for supported workspace and Work Item events.
- The system sends email notifications for supported events, including workspace invitations.
- Users can view, mark as read, and clear notifications.
- Notifications associated with deleted or inaccessible resources do not navigate users to invalid pages.

2.2 Use Cases

Use Case 1: Create Workspace and Invite Members

- Primary Actor: Authenticated User
- Priority: Essential
- Goal in Context: The user wants to create a new workspace and invite members to begin collaborating
- Preconditions: The user is logged in.
- Scenario:
 1. The user selects the option to create a new workspace.
 2. The system prompts the user to enter a workspace name, an optional description, and optional member invitations.
 3. For each invitation, the user enters an email address or username and selects the member's role.
 4. The user submits the workspace creation form.
 5. The system creates the workspace and assigns the user as its owner.
 6. The system creates the member invitations and sends invitation links by email. Registered users also receive real-time in-app notifications.
 7. Invited users open the invitation link, accept the invitation, and join the workspace with their assigned roles.
- Extensions:
 - 2a. If the workspace name is invalid, the system displays an error and asks the user to correct it.
 - 3a. If an email address or username is invalid, the system displays an error and asks the user to correct it.
 - 5a. If workspace creation fails, the system displays an error and allows the user to retry.
 - 6a. If one or more invitations fail, the workspace remains created and the system informs the user how many invitations failed.
 - 7a. If an invited user does not have an account, the system asks the user to sign up before accepting the invitation.
 - 7b. If an invitation is expired, declined, cancelled, or already accepted, the system displays the corresponding invitation status.

Use Case 2: Create and Organize WorkItems

- Primary Actor: Workspace Member

- Priority: Essential
- Goal in Context: The user wants to create and organize project work using Work Items and parent-child relationships.
- Preconditions: A project exists, and the user has access to its workspace.
- Scenario:
 1. The user opens a project dashboard
 2. The user selects the option to create a new Work Item.
 3. The system prompts the user to enter a title, description, priority, status, start date, due date, and assignees.
 4. The user submits the WorkItem information.
 5. The system creates the Work Item, generates a unique Work Item code, and displays it in the appropriate status column on the project dashboard.
 6. The user may open an existing Work Item and create child Work Items to organize the work hierarchically.
- Extensions:
 - 4a. If the title is missing or invalid, the system displays an error and asks the user to correct it.
 - 4b. If the due date is earlier than the start date, the system displays an error and asks the user to correct it.
 - 4c. If a selected assignee is not a member of the workspace, the system rejects the request.
 - 5a. If Work Item creation fails, the system displays an error and allows the user to retry.

Use Case 3: Plan and Manage a Sprint

- Primary Actor: Project Owner/Admin
- Priority: Essential
- Goal in Context: The user wants to create a workspace sprint, organize selected Work Items from workspace projects, and review their current progress.
- Preconditions: The user is a Workspace Owner or Admin, and the workspace contains at least one project with Work Items.
- Scenario:
 1. The user opens the workspace Sprints page and selects the option to create a sprint.

2. The system prompts the user to enter a sprint name, an optional goal, a start date, and an end date.
 3. The user submits the sprint information.
 4. The system creates the sprint and assigns its lifecycle status according to its scheduled dates.
 5. The user opens the sprint and selects the option to manage its Work Items.
 6. The user selects a project and searches or browses the available Work Items.
 7. The user selects the Work Items to add to or remove from the sprint and saves the changes.
 8. The system displays the sprint's Work Items grouped by project, including each Work Item's current status, priority, and schedule.
 9. Workspace members can open the included Work Items to review or update them through the corresponding project workflow.
- Extensions:
 - 1a. If a user without Workspace Owner or Admin permission attempts to create or modify a sprint, the system prevents the action.
 - 4a. If sprint creation fails, the system notifies the user and allows the user to retry.
 - 7a. If adding or removing Work Items fails, the system notifies the user and allows the user to retry.

Use Case 4: Review Pull Requests with Work Item Context

- Primary Actor: GitHub Repository Contributor
- Secondary Actor: GitHub, AI Project Manager
- Priority: Essential
- Goal in Context: The user wants Prismatic to automatically analyze a pull request and provide an AI-generated review using the related Work Item as project context.
- Preconditions: A Workspace Owner or Admin has connected the GitHub repository to the workspace through the GitHub App.
- Scenario:
 1. The user opens, reopens, updates, or marks a pull request as ready for review in a connected GitHub repository.
 2. The pull request title includes a valid Work Item code, such as PRSM-001.
 3. GitHub sends a signed pull request webhook event to Prismatic.

4. The system verifies the webhook signature and confirms that the repository is connected to an active workspace.
 5. The system extracts the first valid Work Item code from the pull request title.
 6. The system searches for the corresponding Work Item within the connected workspace.
 7. The system creates a queued pull request review agent run and associates it with the resolved Work Item and Project.
 8. For newly opened, reopened, or ready-for-review pull requests, the system posts a GitHub comment indicating that the review has started.
 9. The system retrieves the pull request content, changed files, commits, and diff.
 10. The AI project manager analyzes the pull request diff and generates a review summary with optional line-specific comments.
 11. The system posts the AI-generated review directly to the GitHub pull request.
 12. Workspace members can inspect the review agent run and its execution history from the Agent page.
- Extensions:
 - 1a. If the pull request is a draft, the system ignores the event until the pull request is marked as ready for review.
 - 3a. If the webhook event is unsupported, the system accepts but ignores the event.
 - 4a. If the repository is not connected to an active workspace, the system ignores the event.
 - 5a. If the pull request title does not contain a valid Work Item code, the system does not create a review agent run.
 - 6a. If the code is valid but no corresponding Work Item exists, the review run is not associated with a Work Item and may use the connected Project or Workspace context.
 - 7a. If a review run already exists for the same repository, pull request, and head commit, the system does not create or dispatch a duplicate run.
 - 8a. If the initial GitHub comment fails, the system continues processing the review.
 - 11a. If the pull request head commits changes before submission, the system rejects the outdated review.

Use Case 5: Generate and Monitor Project Work with the AI Agent

- Primary Actor: Workspace Member
- Priority: Essential

- Goal in Context: The user wants the AI agent to transform a feature specification into structured Work Items for the current project.
- Preconditions: The user has access to the project.
- Scenario:
 1. The user opens the Agent page within a project.
 2. The system sets the current project as the target of the agent request.
 3. The user enters and submits a feature specification.
 4. The system creates a feature provisioning request and queues an agent run.
 5. The AI agent analyzes the feature specification, current project backlog, workspace members, jobs, and member workloads.
 6. The AI agent plans and creates a hierarchical set of Work Items within the current project.
 7. The AI agent assigns and schedules generated Work Items when suitable and may update eligible existing backlog items.
 8. The AI agent creates a workspace sprint for the requested feature.
 9. The system updates the agent run status and execution steps as processing continues.
 10. The user monitors the run status and execution graph from the Agent page.
 11. After completion, the user reviews the generated Work Items from the current project.
- Extensions:
 - 3a. If the feature specification is empty or exceeds the maximum length, the system prevents submission.
 - 4a. If the request cannot be queued, the system displays an error message.
 - 6a. If agent execution fails, the system marks the run as failed and preserves it in the run history.
 - 9a. If run history or execution steps cannot be loaded, the system displays an error and allows the user to retry.
 - 10a. The user may cancel an active agent run.

2.3 Non-functional Requirements

Performance:

- Standard user requests should return a response within 2 seconds under normal operating conditions.

- Long-running AI operations should acknowledge the request within 2 seconds and execute asynchronously.

Security:

- All production network communication must use HTTPS.
- User passwords must be securely hashed and must never be stored as plain text.
- The system must enforce authentication and role-based authorization for protected operations.
- Sensitive credentials and API tokens must not be exposed to unauthorized users.

Reliability:

- The system must validate user inputs and provide appropriate responses for invalid requests.
- Failures must not leave projects, Work Items, or AI operations in inconsistent states.
- The system should prevent duplicate processing of repeated AI events and actions.
- Navigation to deleted or inaccessible resources must be handled without displaying invalid pages.

Usability:

- The system must provide clear and actionable error messages.
- Forms must indicate which input requires correction without removing entered values.
- The interface should provide consistent navigation and terminology.
- Core project management workflows should be understandable to users without prior training.

Compatibility:

- The web application must support current desktop versions of Chrome, Safari, and Microsoft Edge.
- The interface must remain usable across common desktop screen sizes.

3. System Architecture

3.1 Overview

Prizmatic is a web-based project and workspace management system. The system is divided into a frontend web application, a backend REST API server, a PostgreSQL database, and optional cloud infrastructure components for email, queueing, and object storage. The main user-facing features are authentication, onboarding, workspace management, project management, project member management, and work item tracking.

The frontend is implemented as a Next.js application that renders the user interface, manages user interaction, stores short-lived authentication state, and communicates with the API server. The backend is implemented as a NestJS application that handles business rules, authentication, authorization, validation, transactional workflows, and database access. PostgreSQL stores relational project data such as users, authentication records, workspaces, workspace members, projects, project members, project jobs, work items, labels, invitation records, and refresh tokens.

Major Components

Component	Languages and Technologies	Responsibility
Frontend	TypeScript 5.9.3, Next.js 16.1.1, React 19.2.3, Tailwind CSS 4.1.18	User interface, routing, client-side interaction, auth session handling, API calls
Backend	TypeScript 5.7.3, NestJS 11, Node.js 22 Docker runtime	REST API, authentication, authorization, validation, business workflows, integrations
Database	PostgreSQL 16 planned target, TypeORM 0.3.20, pg 8.16.3	Relational persistence, transactional data, users, workspaces, projects, work items
Cloud services	Oracle Cloud Infrastructure SDK 2.129.0	Optional email delivery, object storage, OCI Function, and queue processing
Tooling	pnpm 10.26.1, ESLint 9, Prettier 3, Husky	Dependency management, linting, formatting, CI checks, git hooks

Third-Party Components and Intended Usage

Component	Version	Used In	Intended Usage
Node.js	24.12.0 frontend local target, 22 Alpine backend Docker runtime	Full stack runtime	JavaScript and TypeScript runtime environment

pnpm	10.26.1	Package management	Dependency installation and script execution
Next.js	16.1.1	Frontend	App Router, page routing, server/client rendering, API route support
React	19.2.3	Frontend	Component-based UI implementation
TypeScript	5.9.3 frontend, 5.7.3 backend	Full stack	Static typing and maintainable application contracts
Tailwind CSS	4.1.18	Frontend	Utility-first styling system
Radix UI	1.x and 2.x packages	Frontend	Accessible base UI components such as dialogs, dropdowns, checkboxes, labels, and tooltips
TanStack Query	5.90.12	Frontend	API request state, caching, mutations, and loading/error handling
Axios	1.13.2	Frontend	HTTP client for communicating with the NestJS API
Zustand	5.0.9	Frontend	Lightweight local UI state management
Zod	4.3.6	Frontend	Runtime validation for environment variables and structured data
NestJS	11.x	Backend	Modular REST API framework with dependency injection
TypeORM	0.3.20	Backend and database	PostgreSQL connection, transactions, migrations, and query execution
PostgreSQL	16 planned target	Database	Relational storage for users, workspaces, projects, work items, memberships, labels, and tokens

PostgreSQL Driver pg	8.16.3	Backend and database	Node.js PostgreSQL connectivity
Argon2	0.44.0	Backend	Secure password hashing
@nestjs/jwt	11.0.2	Backend	JWT access and refresh token support
Helmet	8.1.0	Backend	Security-related HTTP headers
@nestjs/throttler	6.5.0	Backend	API rate limiting
Swagger	11.2.0 Nest package, 5.0.1 UI package	Backend	API documentation
Google Auth Library	10.6.2	Backend	Google OAuth token verification
Octokit / GitHub OAuth	Octokit 5.0.5, auth-oauth-user 6.0.2	Backend	GitHub OAuth authentication support
OCI SDK	2.129.0	Backend infrastructure	Email delivery, queue, and object storage integrations

Design Patterns and Architectural Style

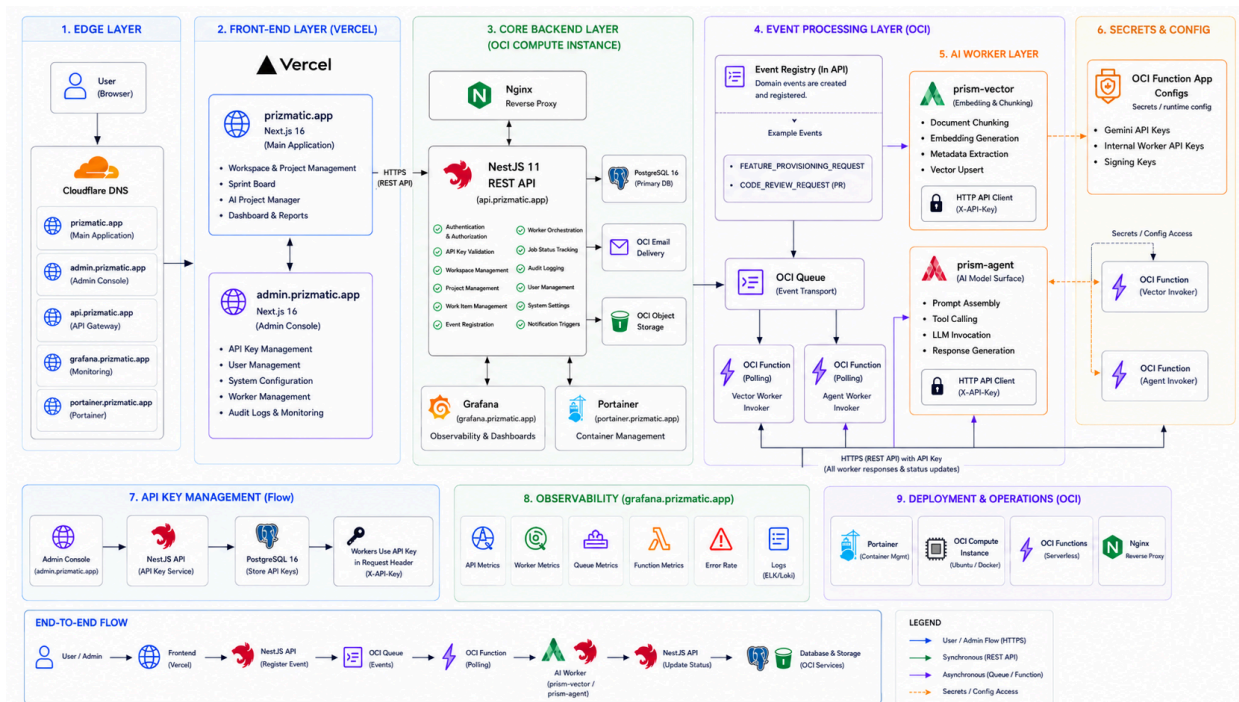
The overall system follows a modular full-stack architecture. The frontend uses a domain-oriented structure and Atomic Design for UI organization. This makes feature code easier to locate while keeping reusable visual components separate from business-specific screens.

It also utilizes a prefetching method, to fully enable the power of server-side-rendering (SSR). Separating server components and client components makes it more resilient in terms of User Experience (UX), as the loading time is much shorter compared to typical client-side-rendering (CSR).

The backend follows a layered clean architecture rather than a strict Model-View-Controller pattern. The main flow is controller to use case to service or repository to database. Controllers handle HTTP transport concerns, use cases coordinate application workflows, services contain reusable business or infrastructure logic, and repositories encapsulate persistence logic.

The project also uses the Repository pattern to isolate database access, Dependency Injection through NestJS providers, DTOs for API request and response contracts, Guards and Interceptors for cross-cutting API behavior, and a Unit of Work pattern for transactional operations.

High-Level Architecture Diagram



The system is designed with a clear separation between edge-level routing, application services, event processing, AI execution, and infrastructure management.

At the edge, Cloudflare DNS handles routing strictly at the subdomain level:

- prismatic.app → Main application hosted on Vercel
- admin.prismatic.app → Admin console hosted on Vercel
- api.prismatic.app → NestJS backend running on OCI
- grafana.prismatic.app → Grafana monitoring dashboard
- portainer.prismatic.app → Portainer container management UI

No path-based routing is handled at the DNS layer.

The frontend consists of two independent Next.js applications deployed on Vercel:

- prismatic.app → Main workspace, project management, sprint planning, dashboard, and AI project management experience

- admin.prismatic.app → API key management, worker management, system administration, and audit monitoring

Frontend applications communicate with the backend exclusively through authenticated REST APIs.

On the backend, services run on OCI Compute instances in a fully containerized environment. Nginx acts as the reverse proxy and ingress layer, forwarding traffic to the NestJS API service. The NestJS API serves as the central orchestration layer responsible for authentication, authorization, workspace management, project management, work item management, event registration, worker orchestration, audit logging, notification triggers, and job status tracking.

The backend integrates with managed services including:

- PostgreSQL 16 (primary database)
- OCI Object Storage
- OCI Email Delivery
- OCI Queue

Prismatic follows an event-driven architecture. When significant domain actions occur, the API registers events such as:

- Feature Provisioning Request
- Code Review Request (PR)

These events are published to OCI Queue, which serves as the asynchronous transport layer between the core application and AI processing infrastructure.

AI workloads are executed in dedicated worker services:

- **prism-vector**
 - Document chunking
 - Embedding generation
 - Metadata extraction
 - Vector indexing
- **prism-agent**
 - Prompt assembly
 - Tool orchestration
 - LLM invocation
 - Response generation

Workers are intentionally separated from the core API and communicate back to the platform through authenticated HTTP requests using API keys managed by the administration console.

OCI Functions act as lightweight queue consumers rather than business-logic services. Dedicated Function invokers continuously poll OCI Queue and dispatch work to the appropriate worker:

- Vector Worker Invoker → prism-vector
- Agent Worker Invoker → prism-agent

Before invoking workers, OCI Functions retrieve required secrets from OCI Function Application Configs, including:

- Gemini API Keys
- Internal worker API keys
- Signing keys

This ensures that secrets remain centrally managed and are never embedded directly in worker deployments.

For observability and operations, the platform includes:

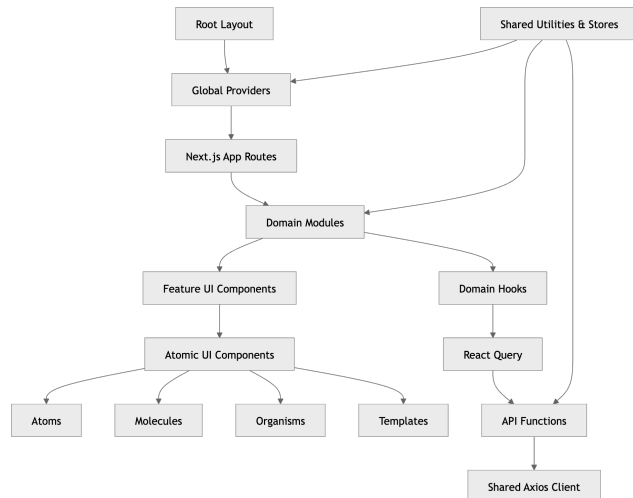
- grafana.prismatic.app for API, worker, queue, function, and system monitoring
- portainer.prismatic.app for container and deployment management

Overall, the architecture separates:

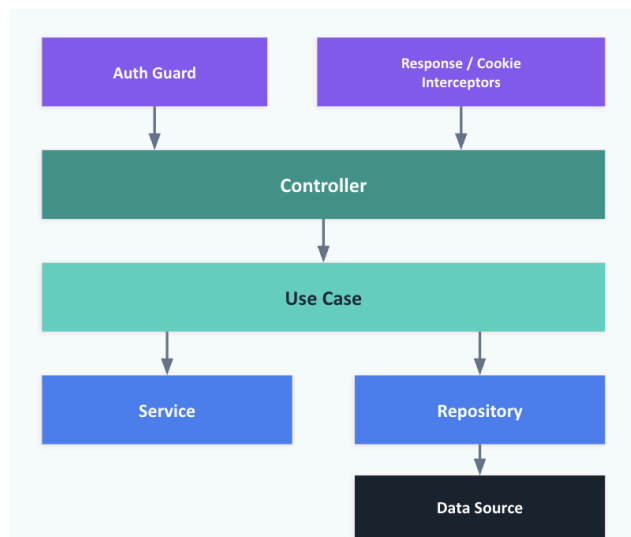
- Edge routing (Cloudflare DNS)
- Frontend delivery (Vercel)
- Core business services (NestJS on OCI)
- Event transport (OCI Queue)
- AI execution (prism-vector and prism-agent)
- Monitoring and operations (Grafana and Portainer)

This structure enables clean service boundaries, asynchronous AI processing, operational flexibility, and independent scaling of application and AI workloads.

Frontend Architecture



Backend Architecture



Technology Stack Rationale

This technology stack was chosen because it provides a consistent TypeScript-based development experience across both the client and server. Using TypeScript on both sides reduces context switching and makes it easier to maintain shared concepts such as users, workspaces, projects, work items, and API payloads.

Next.js is appropriate for the frontend because it supports modern React development, file-based routing, server-side capabilities, and production-ready optimization. NestJS is appropriate for the backend because its module system, dependency injection, guards, interceptors, and validation tools provide a clear structure for a growing API.

PostgreSQL was selected because the system data is relational and requires reliable joins, constraints, transactions, and query performance. TypeORM gives the application a standard integration layer for PostgreSQL while still allowing direct SQL when complex queries are easier to express explicitly.

The selected third-party libraries reduce implementation risk. Radix UI improves accessibility, React Query simplifies remote state management, Argon2 strengthens password security, Helmet and rate limiting improve the API security baseline, and Swagger makes API integration easier. OCI services are included as optional managed infrastructure so the system can support email, queueing, and object storage without building those services from scratch.

AI Agent Architecture

The AI project manager is the platform's defining capability. It runs as an event-driven, queue-coordinated runtime in which each agent run is hydrated with grounded project context, planned by a large language model, and executed as a sequence of approved, idempotent tool actions.

Multi-Agent Orchestration

For complex workflows the runtime decomposes a single objective into a directed acyclic task graph rather than one monolithic plan. An orchestrator expands a workflow's orchestration definition into multiple subagent nodes and an optional synthesizer node. Ready nodes are claimed and dispatched as independent subagent task events; as each subagent completes, a coordinator joins its result and releases any newly unblocked nodes. When all worker nodes finish, the synthesizer node consolidates their outputs into the run's final result. Each subagent plans over only the context scope it requires, which keeps prompts focused and lets independent branches progress in parallel. Workflows that do not require fan-out run as a single-node graph through the same machinery.

Approval and Safety Model

Every tool action is governed by an approval policy with three modes — `auto_commit`, `user_approval_required`, and `suggest_only` — configurable per tool within each workflow. Fully trusted operations, such as creating a sprint or a hierarchical work-item tree during feature provisioning, commit automatically, while higher-impact operations can require explicit user approval or be limited to suggestions. Before an action runs, a validator evaluates it against the

current context and returns one of commit, suggest, replan, or fail; after execution it can request a retry or commit the result. Each action additionally carries an idempotency key that is checked and recorded as it executes, so a redelivered queue message can never apply the same change twice. Together these controls keep the agent's autonomous actions reviewable, recoverable, and safe to run against live project data.

3.2 Data Design

The following data design describes Prismatic's final implemented database structure, as built in the NestJS repositories and DTOs. It has grown well beyond the initial draft to add sprint planning, AI agent execution, semantic search, and GitHub integration alongside the original collaboration core. The database uses PostgreSQL with UUID primary keys and snake_case naming, where entity tables carry an _l suffix and join tables a _map suffix, with the pgcrypto and pgvector extensions enabled for UUID generation and embedding-based semantic search. The application layer maps these fields to camelCase DTOs before returning them to the frontend, automatic schema synchronization is disabled in favor of migrations, and multi-table workflows use the Unit of Work pattern so related changes commit or roll back together.

The schema contains 54 tables grouped into the domains below.

Entity Summary

1. Identity and Access

Table	Purpose
prism_users_l	Stores core user profiles, including email, full name, username, last-active timestamp, and timestamps
prism_user_auths_l	Stores authentication provider records for email, Google, and GitHub sign-in, including password hashes for email auth
prism_user_preferences_l	Stores per-user theme, locale, timezone, and email-notification settings
prism_user_daily_activity_l	Records the dates on which each user was active, for activity tracking
prism_refresh_tokens_l	Stores hashed refresh tokens with expiration and revocation timestamps

prism_email_tokens_l	Stores email verification tokens and their usage status
prism_password_reset_tokens_l	Stores hashed password reset tokens with expiration and usage status

2. Workspace Collaboration

Entity	Description
prism_workspaces_l	Stores workspace metadata, owner, slug, description, timezone, locale, status, work-item code prefix, and soft-delete state
prism_workspace_members_l	Connects users to workspaces with roles owner, admin, member, or viewer
prism_workspace_invitations_l	Stores invitation tokens, receiver email, and role for adding users to workspaces
prism_workspace_invitation_events_l	Records invitation lifecycle events such as sent, accepted, denied, expired, or cancelled
prism_jobs_l	Stores workspace-level job roles that can be assigned to workspace members
prism_workspace_member_job_map	Many-to-many mapping between workspace members and workspace jobs

3. Project and Work Management

Entity	Description
prism_projects_l	Stores projects under workspaces, including slug, description, status, and archive state
prism_work_items_l	Stores typeless work items with a self-referencing parent, priority, status, schedule dates, sequence code, and soft-delete state
prism_work_item_dependencies_l	Stores directed "depends on" relationships between work items
prism_work_item_member_map	Many-to-many mapping between work items and assigned members, with assignee ordering
prism_work_item_labels_l	Stores reusable labels within a project

prism_work_item_label_map	Many-to-many mapping between work items and labels
prism_work_item_comments_l	Stores comments authored on work items
prism_work_item_comment_mentions_l	Records user mentions inside work item comments

4. Sprint Planning

Entity	Description
prism_sprints_l	Stores sprints with goal, start and end times, and status planned, active, closed, or cancelled
prism_sprint_work_item_map	Many-to-many mapping between sprints and the work items in their scope

5. Notifications

Entity	Description
prism_notifications_l	Stores per-recipient in-app notifications with type, actor, target reference, metadata, and read state

6. AI Agent Execution

Entity	Description
prism_feature_provisioning_requests_l	Stores feature provisioning requests dispatched to the agent runtime, with payload pointer, queue message id, and status
prism_agent_runs_l	Stores AI agent runs with trigger type, objective, status, and start/complete timestamps
prism_agent_steps_l	Stores ordered steps within a run, with type, status, input/output summaries, and timing
prism_agent_actions_l	Stores individual tool actions within a run, with status, approval state, and result pointers
prism_agent_action_events_l	Records lifecycle events on agent actions, such as proposed, approved, or executed
prism_agent_memories_l	Stores durable agent memory records that persist plan and context state across runs

7. Documents

Entity	Description
prism_documents_l	Stores uploaded or generated documents with object-storage pointers and optional source work item or comment
prism_document_chunks_l	Stores chunked document content with heading path and token counts for retrieval

8. Semantic Search and Embeddings

Entity	Description
prism_work_item_embeddings_l	Stores 1536-dimension vector embeddings of work item title and description
prism_work_item_comment_embeddings_l	Stores vector embeddings of work item comment bodies
prism_document_chunk_embeddings_l	Stores vector embeddings of document chunks
prism_agent_memory_embeddings_l	Stores vector embeddings of agent memory content
prism_embedding_jobs_l	Queues embedding work for an external vector worker, with attempts, status, and content hash

9. GitHub Integration

Entity	Description
prism_github_installations_l	Stores GitHub App installations linked to a workspace, with account and permission metadata
prism_github_installation_user_grants_l	Records which users have access to a GitHub installation
prism_github_installation_workspace_grants_l	Records which workspaces are granted access to a GitHub installation
prism_github_repositories_l	Stores repositories available through an installation

prism_workspace_repository_links_l	Links a workspace to a connected GitHub repository
prism_project_repository_links_l	Links a project to a workspace-level repository link
prism_github_webhook_events_l	Stores received GitHub webhook events with payload pointer and processing status
prism_pull_requests_l	Stores pull requests with state, branches, head SHA, review summary, and risk level
prism_pull_request_files_l	Stores changed files in a pull request, with status, diff stats, and patch
prism_pull_request_commits_l	Stores commits belonging to a pull request
prism_pull_request_work_item_map	Many-to-many mapping between pull requests and work items, with link source and confidence
prism_pr_review_observations_l	Stores AI review findings on a pull request, with severity, category, location, and suggested patch
prism_github_review_publications_l	Records publication of AI review output back to GitHub, with external ids and status

10. Platform and Operations

Entity	Description
prism_service_accounts_m	Stores internal service accounts used by background workers
prism_service_api_tokens_l	Stores hashed API tokens with scopes, expiration, and revocation for service accounts
prism_admin_audit_events_l	Records administrative audit events with actor, action, and target
prism_object_deletion_queue_l	Queues object-storage objects for deferred deletion with retry tracking

Main Relationships

Users can have multiple authentication records, refresh tokens, workspace memberships, and assigned work items. A workspace belongs to one owner but has many members, and contains projects, job definitions, sprints, and GitHub links. A project belongs to one workspace and contains work items, labels, and documents.

A work item belongs to one project and can optionally reference another work item as its parent, forming a hierarchy without any fixed type such as epic or story. Work items can have multiple assignees and labels through mapping tables, can declare directed dependencies on other work items, can carry comments with user mentions, and can be placed into sprints through the sprint mapping table.

The AI agent domain records autonomous project-manager activity: a feature provisioning request triggers an agent run, which owns ordered steps and individual tool actions, each with its own lifecycle events, while agent memories persist reasoning state across runs. Semantic search is backed by per-entity embedding tables (work items, comments, document chunks, and agent memories) populated by an embedding job queue that an external vector worker drains.

The GitHub domain links installations to workspaces and repositories; pull requests carry their changed files and commits, map to related work items, and accumulate AI review observations that are then published back to GitHub. Workspace invitations connect a sender, receiver, and workspace through an invitation token, with invitation events recording the lifecycle. Refresh, email, and password reset tokens are stored separately from user profiles so authentication state can be revoked or expired without modifying the core user record.

Data Design ERD

<https://github.com/prism-416/.github/blob/main/Documents/ERD.png>

Data Integrity Notes

The system uses UUID identifiers for primary and foreign keys, generated with the pgcrypto extension. Enumerated values are enforced with CHECK constraints: workspace member roles are restricted to owner, admin, member, and viewer; work item priorities to low, medium, high, and urgent; and work item statuses to todo, in_progress, in_review, done, and archived. Work items carry no type column — hierarchy is expressed solely through a self-referencing parent relationship, which is constrained so an item cannot be its own parent.

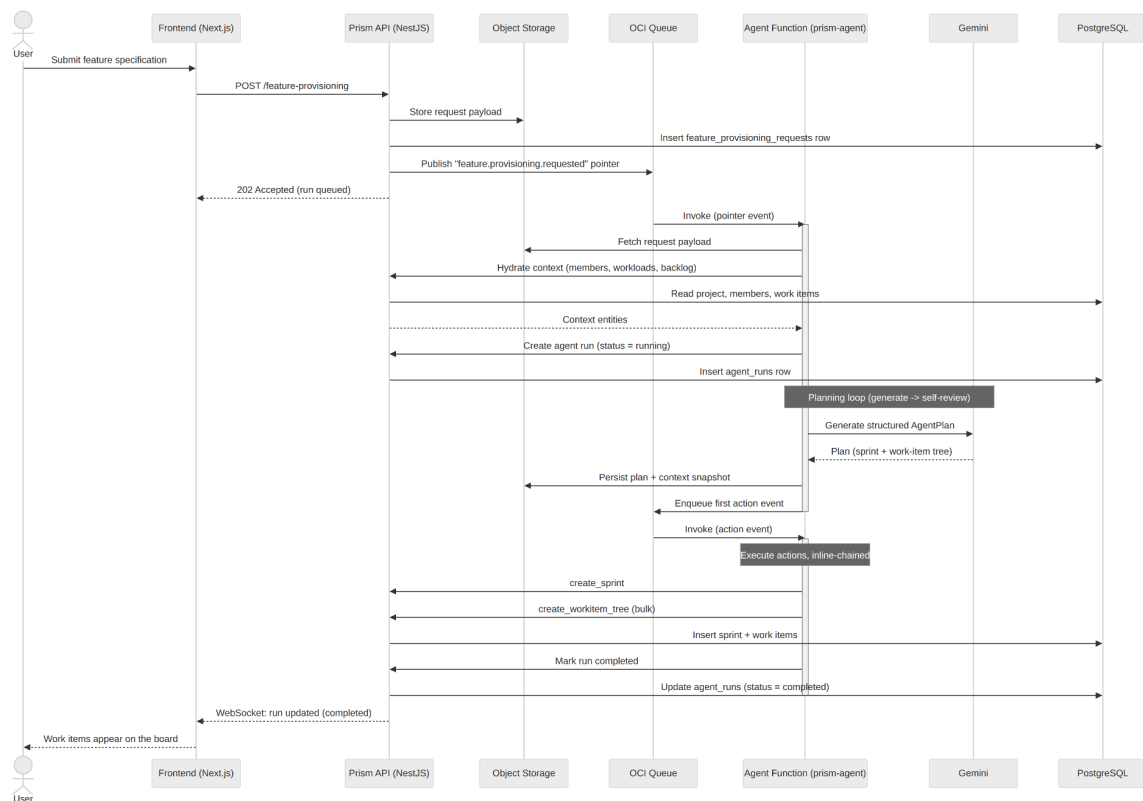
Vector embeddings are stored as vector(1536) columns indexed with HNSW for cosine similarity, and embedding rows cascade-delete with their source entity so search results never

reference deleted content. Soft deletion is used for workspaces, projects, and work items through `deleted_at` and `archived_at` columns rather than physical deletion, and object-storage cleanup is deferred through a deletion queue.

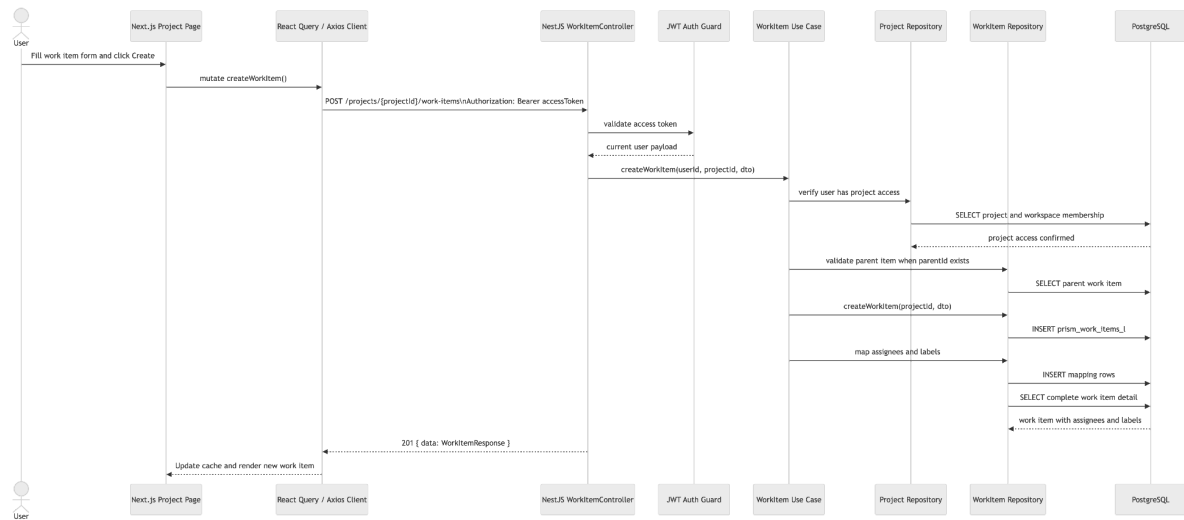
The backend disables automatic TypeORM schema synchronization and manages all schema changes through versioned migrations. Application workflows that modify multiple tables use the Unit of Work pattern so related database changes are committed or rolled back together.

3.3 UML Sequence Diagrams

Use Case 1: AI agent running workflow



Use Case 2: Create Work Item in a Project



3.4 API Design

<https://docs.google.com/spreadsheets/d/1rcqHgPI2zq2jGlvbYypsUSPhWkkgH1LknW6GgSyOII/edit?usp=sharing>

3.5 Deployment

The frontend is deployed as a Next.js application on Vercel. Vercel is a good fit because it provides built-in support for Next.js routing, server-side rendering, API routes, environment variables, preview deployments, and HTTPS. The frontend uses environment variables such as NEXT_PUBLIC_API_HOST and NEXT_PUBLIC_SITE_URL to connect to the deployed API server.

The backend is deployed as a Dockerized NestJS service on an OCI Compute instance behind HTTPS. The repository includes a multi-stage Dockerfile using Node.js 22 Alpine, and the deployment target runs the built server with node dist/src/main.js. Required environment variables include database credentials, JWT secrets, CORS origins, OAuth client settings, and OCI integration settings.

The database runs as a managed PostgreSQL 16 database. A managed database is preferred over hosting PostgreSQL manually because it simplifies backups, monitoring, upgrades, and recovery. The NestJS server connects to it through TypeORM using PG_HOST, PG_PORT, PG_USER, PG_PASSWORD, PG_DB, and optional PG_SCHEMA.

Infrastructure services use Oracle Cloud Infrastructure because the backend integrates the OCI SDK. OCI Email Delivery handles email verification and workspace invitation emails, OCI Object Storage stores uploaded files and generated assets, and OCI Queue provides the asynchronous transport layer between the core application and the AI processing workers.

GitHub Actions is used for continuous integration. The frontend checks run pnpm lint, pnpm type-check, and pnpm build. The backend checks run pnpm run lint:check, pnpm run test, and pnpm run build. Deployment secrets are stored in the deployment platform rather than committed to the repository.

3.6 Code Conventions

The project follows TypeScript conventions enforced by ESLint and Prettier. The frontend uses Next.js ESLint rules and TypeScript checks, while the backend uses typescript-eslint, recommended ESLint rules, and Prettier. The team also uses the Airbnb JavaScript Style Guide as the general JavaScript/TypeScript style reference: <https://github.com/airbnb/javascript>. Additional tooling references are TypeScript ESLint at <https://typescript-eslint.io/> and Prettier at <https://prettier.io/docs/>.

Naming Conventions

Item	Convention	Example
TypeScript variables and functions	camelCase	createWorkspace, accessToken
React components	PascalCase	Sidebar, MainLayout
Classes and DTOs	PascalCase with descriptive suffixes	CreateWorkspaceDto, ProjectRepository, AuthUseCase
Hooks	use prefix with camelCase	useApiQuery, useCurrentUser
Constants	UPPER_SNAKE_CASE	WORKSPACE_MEMBER_ROLES
Database tables and columns	snake_case	prism_workspaces_l, workspace_id
API URLs	plural RESTful resource names	/workspaces/:workspaceId/members
File names	existing project style by layer	workspace.controller.ts, work-item.usecase.ts, Button.tsx

Formatting Conventions

Frontend formatting follows prism-nextjs/.prettierrc.json: 2 spaces, no tabs, 120-character print width, double quotes, trailing commas, and no parentheses around single-argument arrow functions when possible. Backend formatting follows prism-nextjs/.prettierrc: single quotes, trailing commas, and LF line endings.

Code should be organized by responsibility. Frontend feature code should stay under src/domains, reusable UI should stay under src/atomics, and cross-feature utilities should stay under src/shared. Backend routes should stay in controllers, workflow logic in use cases, reusable domain or infrastructure behavior in services, database access in repositories, and request/response contracts in DTOs.

Comments should be used only when they clarify non-obvious business rules, security behavior, or complex queries. The team should avoid comments that simply repeat what the code already says. API and database changes should be reflected in DTOs, Swagger annotations, and this API design section.

Style Checking

The frontend should be checked with:

- pnpm lint
- pnpm type-check
- pnpm build

The backend should be checked with:

- pnpm run lint:check
- pnpm run test
- pnpm run build

Layering Conventions

The system follows a strict layering convention to enforce separation of concerns, scalability, and long-term maintainability across both frontend and backend.

At a high level, the architecture is divided into three major areas: domains, shared, and presentation layers.

The core business logic lives inside domain boundaries. Each domain is treated as an isolated unit that owns its own components, API logic, types, and internal workflows. Cross-domain dependencies are intentionally restricted. A domain must not directly depend on another domain. If logic needs to be reused, it is extracted into a shared layer instead. This prevents hidden coupling and keeps the system composable as it grows.

The shared layer acts as a foundation. It contains domain-agnostic utilities such as query abstractions, hooks, constants, and low-level helpers. This layer is strictly dependency-safe,

meaning it does not depend on any domain or UI layer. It exists purely to support reuse without introducing circular dependencies.

On the presentation side, the UI is structured using an Atomic Design approach. Components are organized by composition depth rather than feature ownership. Lower-level primitives like atoms are kept completely generic, while higher-level components progressively compose them into more complex structures. Importantly, components always move upward in the hierarchy based on their dependencies, never sideways. This prevents ambiguous ownership and enforces a predictable UI structure.

Any UI component that contains domain-specific knowledge is explicitly placed inside its corresponding domain instead of the shared UI layer. This ensures that business context does not leak into generic presentation code.

Data flow follows a server-first model. Pages are treated as thin entry points, delegating data fetching to server components and passing the result down into a single client boundary. The client layer consumes this data through a standardized query abstraction, ensuring consistency in caching and state management. This approach removes the need for complex hydration strategies and keeps data flow explicit and predictable.

On the backend, the same principle of strict layering applies. Each feature module encapsulates its own controllers, use cases, services, and repositories. Controllers remain thin and handle only HTTP concerns. Use cases orchestrate business logic and define transaction boundaries. Repositories own persistence and SQL. Services handle side effects and external integrations. This layered separation ensures that business logic is not fragmented across the system and remains testable and maintainable.

Overall, this layering convention enforces a few critical invariants: business logic is isolated, dependencies flow in one direction, and shared abstractions remain truly reusable.

4. Schedule

The project was developed through four primary milestones, followed by an additional integration and stabilization period. Some components implemented during the earlier milestones were later refactored or replaced as the system architecture and product requirements evolved.

Project Milestone 1 (5/11/2026)

We focused on establishing the core product foundation. The goal of this milestone was to deliver a functional application where users could navigate projects and interact with Work Items. This included implementing the project detail page, Work Item detail page, and basic

Work Item CRUD operations. Heejoon Yi developed the Project API, corresponding frontend pages, and task management UI. In parallel, Jeongmin Sohn implemented supporting backend components, including Work Item-related APIs, database entities, and related endpoints, while supporting frontend- backend integration. These efforts established a complete end-to-end user flow without relying on placeholder logic.

Project Milestone 2 (5/13/2026)

We introduced collaboration features and real-time capabilities. Sprint functionality was implemented, including a board-based workflow for managing Work Items. Heejoon Yi focused on implementing the sprint board and its related UI. Jeongmin Sohn implemented the WebSocket layer and its client-side integration to enable real-time updates across clients. Relevant project state changes were synchronized in real time, establishing the foundation for collaborative interactions.

Project Milestone 3 (5/20/2026)

We focused on developing the agent system and integrating it with the product's project management workflows. Heejoon Yi led the implementation of the event-driven agent runtime and built the vector processing pipeline for document chunking, embedding generation, and pgvector-based similarity search. Jeongmin Sohn focused on stabilizing the application layer required for agent-driven operations, resolving Work Item status conflicts that prevented planning actions from being applied reliably, and improving frontend-backend integration. He also redesigned the project overview into a functional dashboard, streamlined Work Item and sprint creation flows, and improved sprint status visualization. Together, these efforts established the technical foundation for executing and presenting agent-generated project management actions within the application.

Project Milestone 4 (5/27/2026)

We focused on completing the product's core collaboration features and preparing its key differentiating features. Heejoon Yi developed the foundation for the GitHub PR review bot and its integration with the agent system. Jeongmin Sohn implemented supporting collaboration features, including comments, notifications, and real-time system interactions.

In addition, Heejoon Yi handled production deployment by implementing an automated CI/CD pipeline using GitHub Actions. The pipeline was configured to trigger deployment whenever

changes were merged into the designated deployment branch. For the backend, the deployment process included building a Docker image, pushing it to GitHub Container Registry, pulling the latest image onto the OCI Compute instance, restarting services through Docker Compose, and performing health checks. The frontend was automatically deployed through Vercel's Git-based integration.

Final polishing and system integration were performed collaboratively to provide a cohesive and deployment-ready user experience.

After Project Milestone 4

The team focused on completing key integrations, expanding the agent's capabilities, and improving production stability. Heejoon Yi completed the GitHub PR review workflow, including webhook-based review triggers, repository linking, Work Item code resolution, and automated review comments. He also strengthened the production agent infrastructure through multi-agent orchestration, durable state management, structured logging, monitoring, and performance improvements. Jeongmin Sohn expanded the agent's project management capabilities by implementing backlog refinement, sprint planning, hierarchical Work Item generation, semantic duplicate detection, and reliable Work Item assignment. He also integrated manual and scheduled agent workflows with the queue and added real-time agent run updates and execution history to the frontend. During final integration testing, he identified and resolved issues involving agent execution, authentication, notifications, deleted-resource navigation, and frontend-backend synchronization.

During this final stage, both team members collaboratively resolved integration issues across the frontend, backend, agent runtime, GitHub integration, and cloud infrastructure. These improvements transformed the milestone implementation into a more reliable and cohesive application suitable for final demonstration and deployment.

Future Works

Future development will focus on extending Prismatic from an AI-assisted project management platform into an active development collaboration environment. The team plans to integrate AI coding workers, including Codex and Claude, through the Model Context Protocol (MCP). These AI workers will participate as workspace members, receive Work Item assignments,

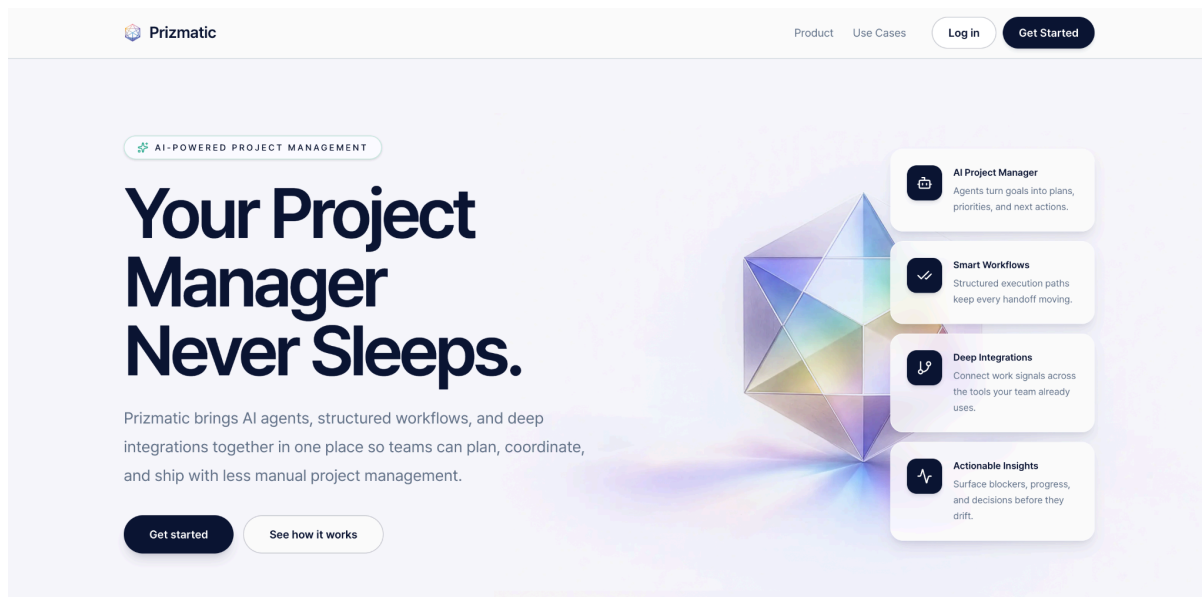
perform development tasks, and collaborate with human members under the coordination of the AI project manager.

The team has already recruited software developers and small development teams who have agreed to use Prismatic and provide feedback based on real project workflows. Their feedback will be used to identify usability issues and guide improvements to the overall UI/UX, project management workflows, and agent interactions.

Based on this real-world validation, the team aims to develop a more reliable and production-ready AI project manager by improving agent accuracy, security, scalability, observability, and human approval controls. The long-term goal is to continue validating Prismatic with real users and prepare the platform for release as a commercial product.

5. Demo Screenshots

Home Page



Create a new workspace

Create a new workspace

Organize projects, invite teammates, and keep everything in one calm place.

Workspace name 4/20

Description (optional) 0/1000

Invite members (optional)

Search by name, username, or email, then select a member to add.

Member

Add

Members can collaborate on workspace content without administrative permissions.

jeongmin_sohn

jameszanga@gmail.com

Existing

Member

×

user

jeongmin.sohn@stonybrook.edu

Existing

Member

×

Cancel

Create workspace

In-app Invite Notification

P

/ Workspaces

All workspaces

Workspaces

Workspaces

Create, organize, and switch between the workspaces your teams use.

Notifications

1 unread ✓ Read all 🗑 Clear all

- Workspace invitation**

You have been invited to join Test as member.
less than a minute ago

Email Invite Notification

☰

Gmail

Compose

Inbox

Starred

Snoozed

Sent

Drafts

Trash

Categories

Purchases

More

Labels

Search mail

Active

1 of 1,494

←

🗑

🕒

📧

🕒

🔄

📁

📄

⋮

Invitation to join Test

External

Inbox x

Prismatic

noreply@prismatic.app

7:02 PM (2 minutes ago)

Caution: The sender domain (prismatic.app) was only recently created, which may indicate it is a malicious domain. Verify the email's authenticity with the sender using your organization's trusted contact list before replying or taking further action.

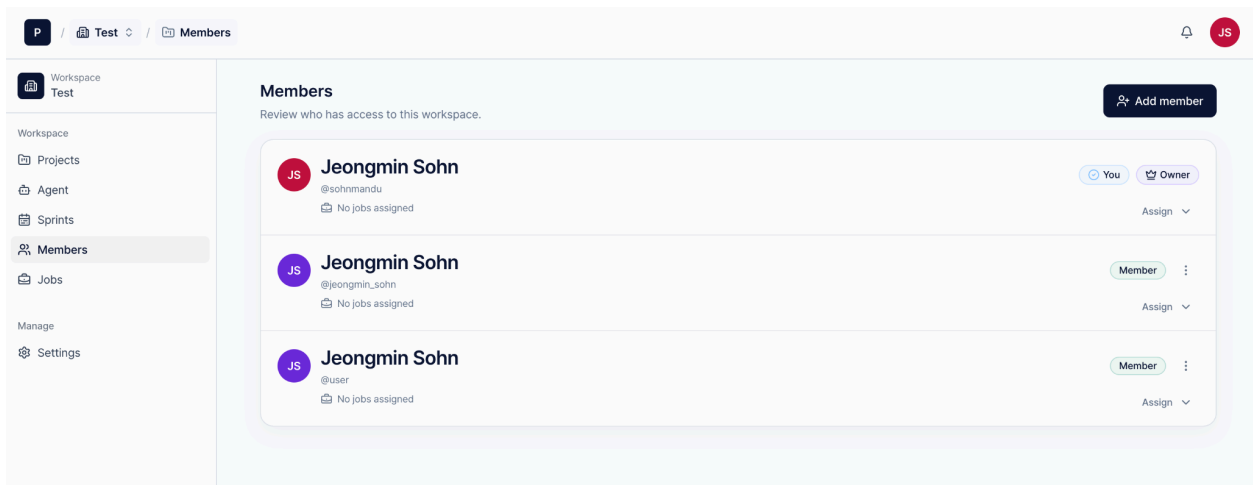
You have been invited to join Test as member. Review the invitation here: <https://protect.checkpoint.com/v2/r01/...> <https://dev.prismatic.app/workspaces/invitations/accept?token=aba7c925-0df1-4df8-8840-5c5b9eee0535...>

↩ Reply

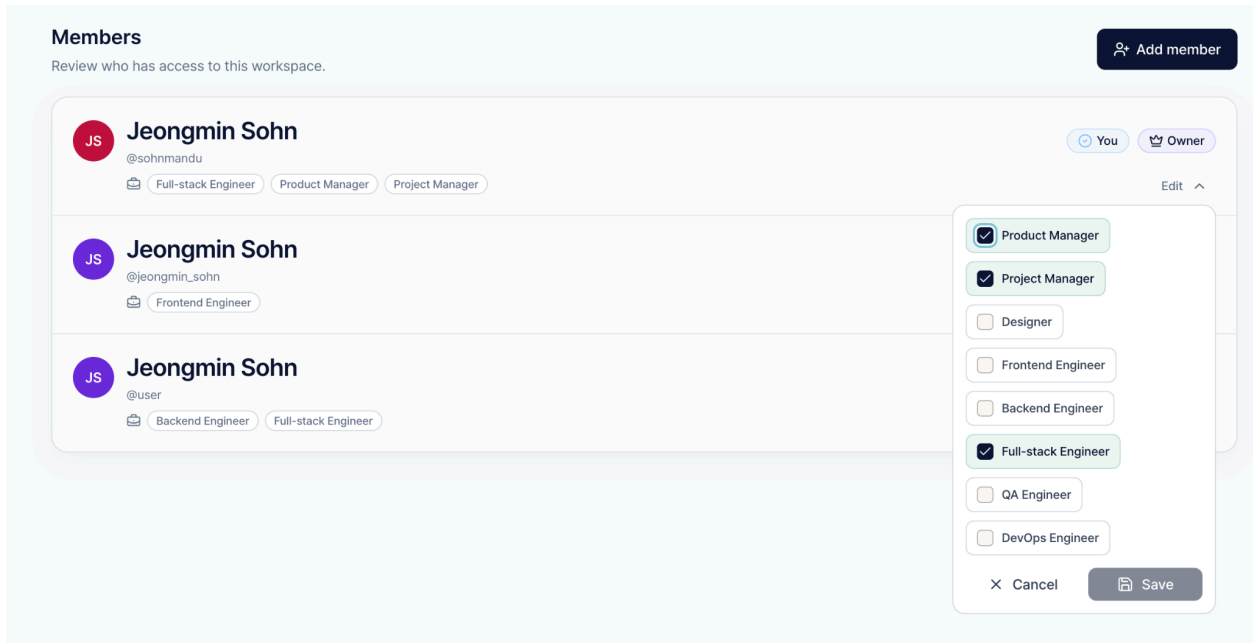
➡ Forward

😊

Members



Member Job Assignment



Run AI Agent



Agent overview

Review recent agent runs, inspect each execution graph, and start a new task-generation run.

Agent run history

Recent runs stay visible after completion, with the latest execution graph shown first.

Completed

project_manager

Hide graph

Turn the requested feature specification into one sprint and one complete, hierarchical work item breakdown with thoughtful assignee selection, committed in as few actions as possible.

Created

Started

Finished

Queue wait

Duration

Jun 12, 2026 19:17:44

Jun 12, 2026 19:17:48

Jun 12, 2026 19:21:52

4s

4m 5s

Trigger event

Execution graph

Request created

Jun 12, 2026 19:17:44

Turn the requested feature specification into one sprint and one complete, hierarchical work item breakdown with thoughtful assignee selection, committed in as few actions as possible.

Completed

Step 1 Plan Jun 12, 2026 19:21:03

Planned 4 action(s).

Create the sprint for the External Notification Channels feature.

Completed

Step 2 Execute Jun 12, 2026 19:21:09

Completed action sprint_1

Provision the full work item breakdown for the External Notification Channels feature.

Completed

Step 3 Execute Jun 12, 2026 19:21:40

Completed action tree_1

Increase priority of existing foundational data and fan-out tasks to support new external delivery features.

Completed

Step 4 Execute Jun 12, 2026 19:21:46

Completed action bulk_1

Propose rebalancing workload from overloaded members.

Completed

Step 5 Execute Jun 12, 2026 19:21:50

Completed action sugg_1

Results - work items

TEST-028

Implement GET /notifications/unread-count

Create a highly optimized route returning just the total unread...

Jun 15, 2026 ~ Jun 16, 2026

Todo Medium

26

TEST-018

Implement websocket push for new notifications

Wire up the notification creation to push new notifications to...

Jun 14, 2026 ~ Jun 15, 2026

Todo Medium

26

TEST-017

Implement notification fan-out and deduplication logic

Create an internal service triggered by work item and sprint...

Jun 14, 2026 ~ Jun 15, 2026

Todo Medium

26

TEST-012

Implement GET /notifications

Create the list route for a user's notifications. Support...

Jun 15, 2026 ~ Jun 16, 2026

Todo Medium

26

TEST-016

Implement PUT /notifications/id/read and unread toggle

Create a route to toggle the read status of a specific...

Jun 16, 2026 ~ Jun 17, 2026

Todo Medium

26

TEST-015

Implement PUT /notifications/read-all

Create a route to mark all unread notifications as read for the...

Jun 16, 2026 ~ Jun 17, 2026

Todo Medium

26

TEST-013

Implement GET and PUT /workspaces/id/preferences/notifications

Create routes to fetch and update the user's notification...

Jun 17, 2026 ~ Jun 18, 2026

Todo Medium

26

TEST-014

Implement GET /projects/id/activity-feed

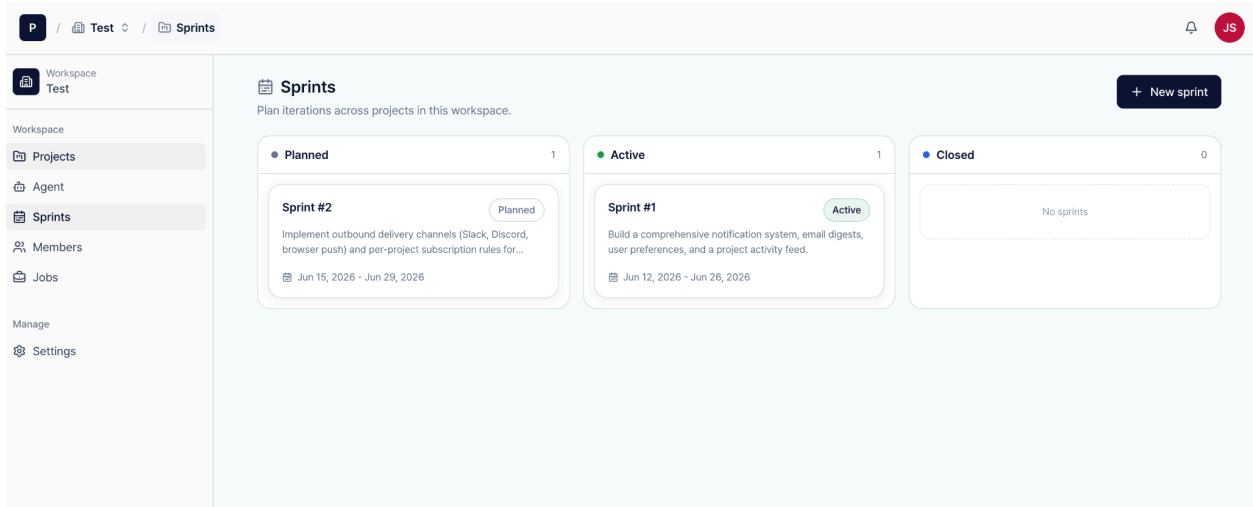
Create a paginated (cursor-based) route returning raw event...

Jun 17, 2026 ~ Jun 18, 2026

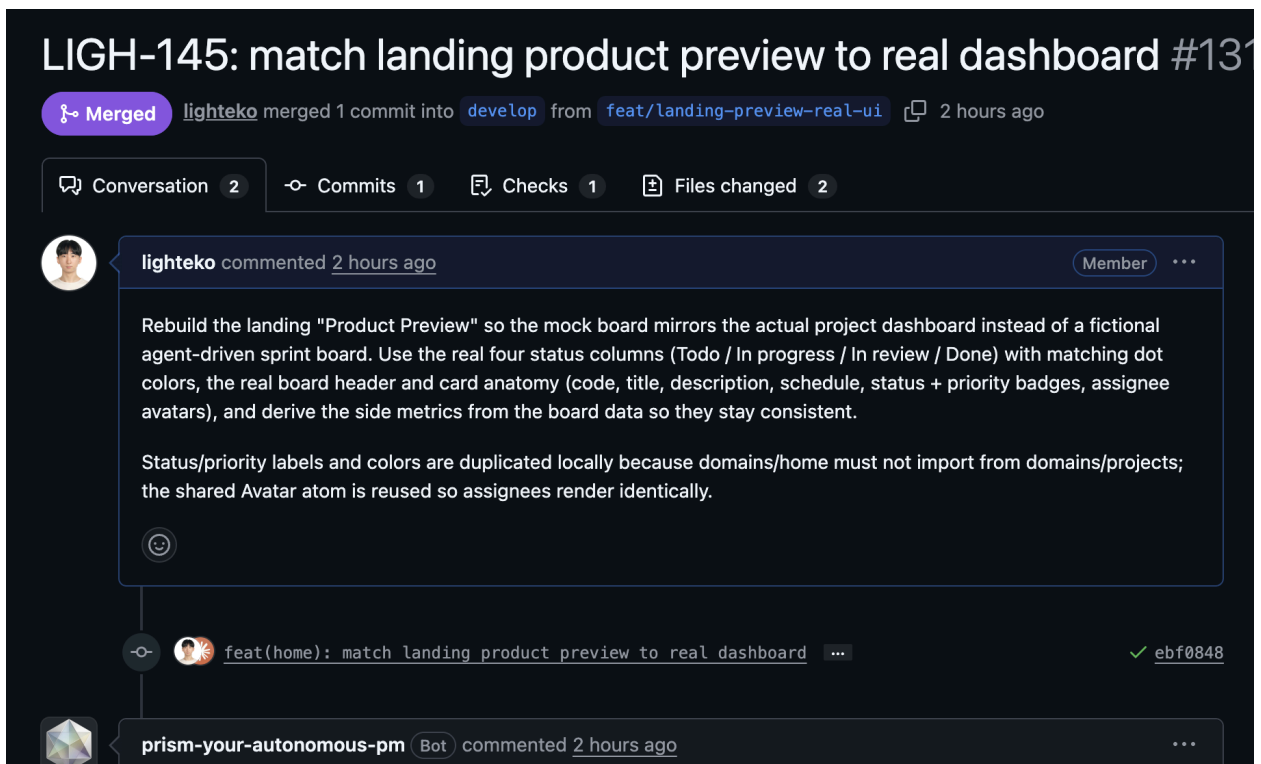
Todo Medium

26

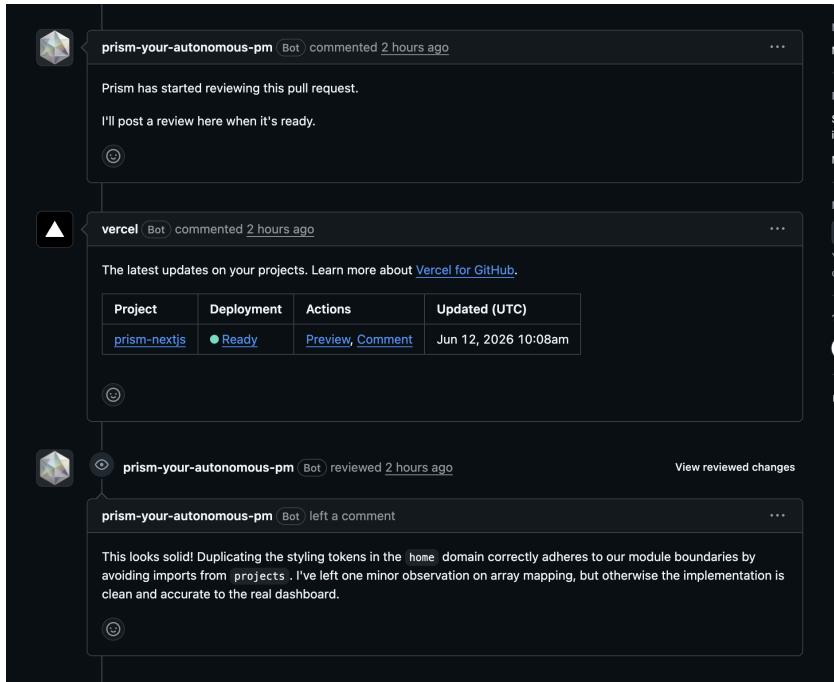
Results - Sprint update



Run PR review bot



User created PR



Moments later, Prism Agent comments on the PR informing the user that the review process has begun.

1~2 minutes later, the review comment appears.

Completed Pull Request Review Hide graph

Review GitHub PR #131 for prism-416/prism-nextjs. Project: 89edeef5-3f4e-45ae-b9d8-74088677759e. Head SHA: ebf0848d87c0096308bbf2de2fc84149e92034e.

Created	Started	Finished	Queue wait	Duration
Jun 12, 2026 19:08:35	Not started	Jun 12, 2026 19:12:55	Not started	Not started

Trigger: Webhook Work item: LIGH-145 — Data Model & Infrastructure

Execution graph

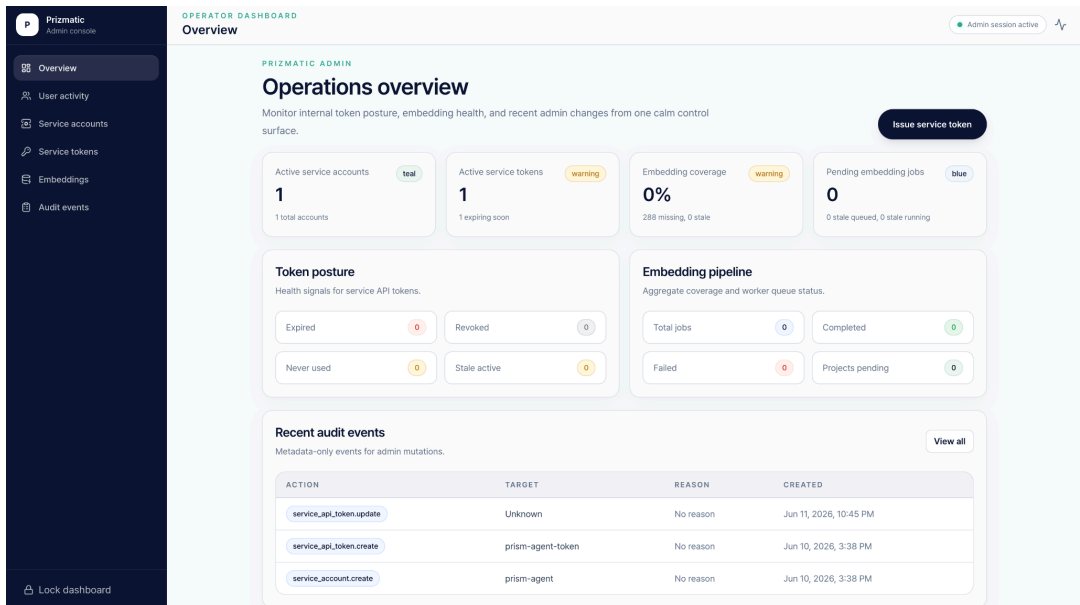
- Request created**
Jun 12, 2026 19:08:35
- Inspect the hydrated pull request diff and produce a helpful, line-anchored code review when a pull request is opened, updated, or review is requested.** Completed
Step 1 Plan Jun 12, 2026 19:12:08
Planned 1 action(s).
- Submit code review for pull request 131** Completed
Step 2 Execute Jun 12, 2026 19:12:53
Completed action submit_review

The internal plans and works are visible in the agent panel.

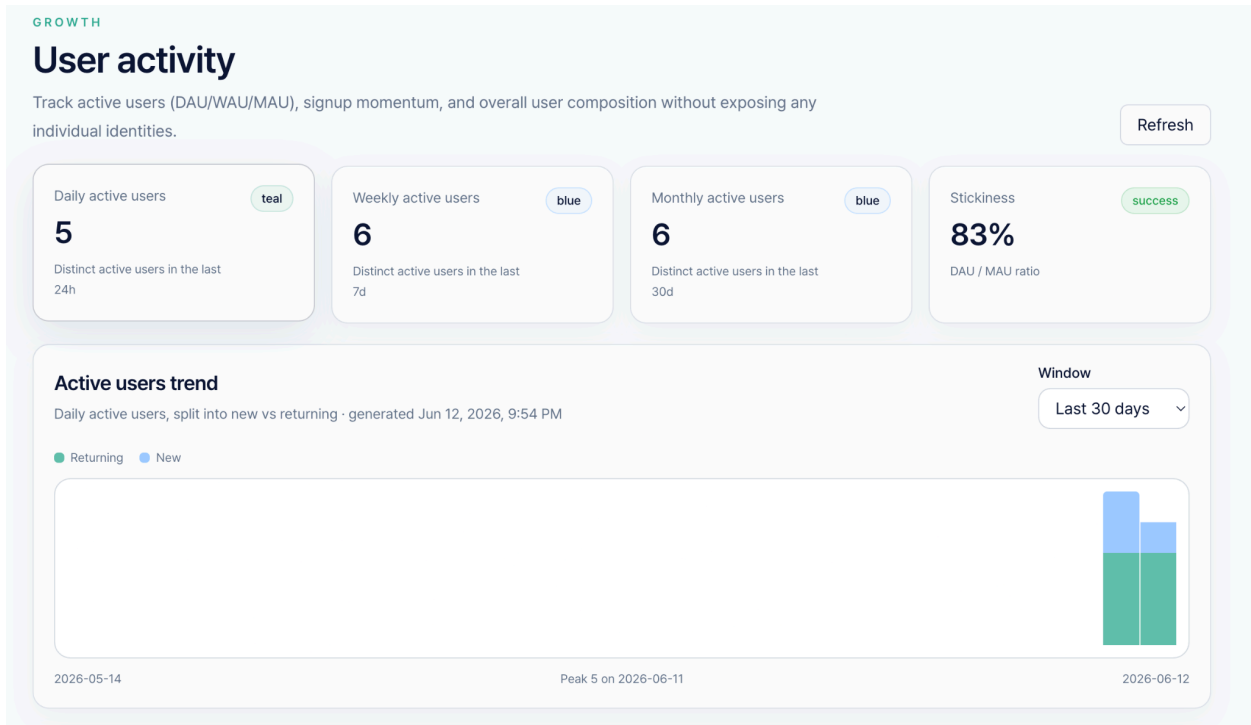
Admin Panel

Website: admin.prismatic.app

Overview Page



User activity Page



Internal API token generation page

INTERNAL API

Service tokens

Search, issue, update, and revoke internal service API tokens without exposing token hashes.

+ Issue token

Total tokens

1

blue

Active

1

success

Expiring soon

1

Within 30 days

warning

Stale active

0

Unused for 90 days

danger

Health thresholds

These controls use the documented token health query parameters.

Expiring within days

30

Stale after days

90

Refresh

Token inventory

Filter by the documented token search fields. Raw token values are never returned in inventory.

Service name

embedding-worker

Service account ID

UUID

Status

All statuses

Expires before

연도. 월. 일. -- --:--

Last used before

연도. 월. 일. -- --:--

Refresh

Clear

TOKEN	SERVICE	STATUS	EXPIRES	LAST USED	ACTIONS
prism-agent-token prism_it_Gep5ag68cDBN05yh	prism-agent active account	active	Jul 9, 2026, 9:38 PM	Jun 12, 2026, 9:20 PM	Edit Revoke

Service account scope settings

Edit service token

Token name

prism-agent-token

Scopes

☒ Agent workers
agents:invoke
Read run state and write agent run status, steps, actions, and action events.

☒ Document readers
documents:read
Read project document metadata and content needed by internal services.

☒ Document writers
documents:write
Upload documents, append chunks, and write document chunk embeddings.

☐ Embedding workers
embeddings:write
Claim embedding jobs, update job status, and write work item embeddings.

☒ Project readers
projects:read
Search work items, read work item detail, and list child work items.

☒ Project writers
projects:write
Create, update, delete, and reorder work items for internal workflows.

☒ Pull request reviewers
pull_requests:write
Read pull request content and create pull request reviews for internal workflows.

☒ Repository readers
repositories:read
Read connected repository metadata needed by internal GitHub workflows.

☒ Sprint writers
sprints:write
Create and update sprints, delete sprints, and manage sprint work items.

Expires at

2026. 07. 09. 오후 12:38

Audit reason